NAModule1NetworkXTutorial

October 12, 2026

1 Module 1 NetworkX Tutorial

In this tutorial, we will show examples of several tools you may find helpful for this module's programming assignment.

In this module, you will be working with an undirected graph representing the American football season of the year 2000, where nodes are teams and we form an undirected edge between teams that have played against each other at least once during the season. Below is an example of a node in the dataset:

```
node [
    id 1
    label "FloridaState"
    wins 11
    losses 2
    conference 0
]
```

As you can see, each node has an ID, a 'label' variable representing the name of the college it represents, the number of wins and losses it had in the season, and its 'conference.' In American Football, conferences are groups of teams that are in a league together, and play against each other to win the league title. It is not uncommon however, for teams to play teams that are outside of their league.

Below, we'll import some necessary libraries and read the graph.

[7]:
```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import adjusted_rand_score


G_football = nx.read_gml('../../../assignments/source/NAModule1Assignment/
 ↪assets/football1.gml', label='id')
```

2 Accessing a node in the graph

In this module's assignment, you may use functions that give you a set of node ids. You will need to access these nodes in the graph by their `id`, and obtain features such as their `label` or `win`

count.

Let's say we have node id 1, and we want to access its 'label' field to get the name of the node's college.

We first access all the nodes in G_football with `.node` and then subscript into it with [1], to obtain a dictionary containing node 1's information. We can provide a key to this with `[]` as well. Below, we access the 'label' field of the node.

[6]:
```
G_football.nodes[1]['label']
```

[6]:
```
'FloridaState'
```

3 Computing Correlation Coefficients

In this module's programming assignment, you may need to measure the correlation between the values of two equally sized lists. Below, we provide an example of how this is done.

[ ]:
```
X = [1, 2, 3, 4, 5]
y = [2, 4, 6, 8, 10]

correlation_matrix = np.corrcoef(X, y)
correlation_coefficient = correlation_matrix[0, 1]
print(f'Correlation Coefficient: {correlation_coefficient}')
```

4 Computing Assortativity Coefficients

In this module, you learned about attribute, degree, and numeric assortativity measurements and what they represent. In this module, you will need to compute these measurements. Below, we provide examples of how you could obtain these metrics.

4.0.1 Attribute Assortativity

Below is a line which gives the attribute assortativity coefficient, giving us a coefficient representing the tendency of nodes with the same attributes to form edges between one another. In this case, since we select "conference," we obtain a measure representing the tendency of universities in the same conference to play one another.

[ ]:
```
nx.attribute_assortativity_coefficient(G_football, attribute="conference")
```

[ ]:
```
0.6275381679111909
```

4.0.2 Numeric Assortativity

Below is a line which gives the numeric assortativity coefficient, giving us a coefficient representing the tendency of nodes with similar numeric attributes to form edges with each other. In the context of our football dataset, as we've selected the 'wins' attribute, the coefficient tells us the tendency of teams with similar win counts to play against other.

[ ]:
```
nx.numeric_assortativity_coefficient(G_football, attribute="wins")
```

[ ]:
```
-0.049806582644503085
```

4.0.3 Degree Assortativity

Below is a line which gives the degree assortativity coefficient, giving us a coefficient representing the tendency of nodes with similar degrees to form edges between one another. In the context of our football dataset, this tells us the tendency of teams to play other teams with a similar number of total games played. This coefficient is actually just the numeric assortativity coefficient, but with the 'degree' as the numeric argument.

[ ]:
```
nx.degree_assortativity_coefficient(G_football)
```

If we were working with a directed graph, we could specify the 'x' and 'y' arguments used when calculating the coefficient as the in-degree or out-degree of each node. Below is code that creates a small directed graph and sets the 'x' and 'y' arguments.

[25]:
```
G = nx.DiGraph()

edges = [
    ("1", "2"),
    ("2", "3"),
    ("3", "1"),
    ("3", "4"),
    ("4", "2")
]

G.add_edges_from(edges)

in_degree_assortativity = nx.degree_assortativity_coefficient(G, x='in', y='in')

out_degree_assortativity = nx.degree_assortativity_coefficient(G, x='out',␣
 ↪y='out')

out_in_assortativity = nx.degree_assortativity_coefficient(G, x='out', y='in')

print(f"In-Degree Assortativity: {in_degree_assortativity}")
print(f"Out-Degree Assortativity: {out_degree_assortativity}")
print(f"Out-In Degree Assortativity: {out_in_assortativity}")
```

```
In-Degree Assortativity: -0.4082482904638645
Out-Degree Assortativity: -0.4082482904638645
Out-In Degree Assortativity: -0.6666666666666686
```

5 Structural Holes

5.0.1 Constraint

In this module's assignment, you will be working with the dictionary you obtain from `nx.constraint`.

The function returns a dictionary, where each key is a node `id` and the value is the constraint coefficient.

In the context of our dataset, lets say we have a college 'A'. The coefficient corresponding to A in the dictionary measures how invested/connected college A's neighbors are in nodes that are also connected to college A. If college A has a low coefficient, we might see it as a broker spanning structural holes, which you learned about in this module's lectures.

Remember, you can loop through each value in a dictionary with `.items()`, as shown below.

[ ]:
```
constraints = nx.constraint(G_football)

for node_id, constraint_value in constraints.items():
    print(f'{node_id} : {constraint_value}')
```

```
<class 'dict_items'>
0 0.1586688769711707
1 0.1843312066370778
2 0.15886454507193143
3 0.15275783402203855
4 0.17942117982563766
5 0.16208239185485607
6 0.14832231723803696
7 0.1684377869605142
8 0.18895464790656374
9 0.19116700437887518
10 0.19056019283746561
11 0.16851853446076936
12 0.1713481086113662
13 0.1671528394387147
14 0.18007545658606267
15 0.15483155417814512
16 0.17938519302719158
17 0.1636418277440066
18 0.17931127327647134
19 0.18591357318489177
20 0.16690390000683014
21 0.1869044691847096
22 0.19868485759169455
23 0.1866161890210225
24 0.15073731251913075
25 0.20558149109426346
26 0.17265228558518603
```

27 0.16804585634287617
28 0.18398266238894742
29 0.1980083327641555
30 0.1944149989754798
31 0.17266412349557855
32 0.16732406711745557
33 0.24121673553719014
34 0.17516330294155777
35 0.1862871388566355
36 0.15141666128615705
37 0.20986011504982205
38 0.15836940234636204
39 0.17427469662819023
40 0.16306819130447983
41 0.20992357667584946
42 0.16765390453702142
43 0.15587009556672987
44 0.16622361860528656
45 0.20986011504982205
46 0.20069638913553267
47 0.16623978325706348
48 0.1668561270101466
49 0.20555764254111367
50 0.19353270830262972
51 0.19329606272255911
52 0.18361271808999088
53 0.1803172189062341
54 0.2036628099173554
55 0.18147872413086535
56 0.1866735537190083
57 0.17791157024793391
58 0.1257241289154168
59 0.1691948176923911
60 0.16622839969947406
61 0.16110033467659315
62 0.16627279557407282
63 0.14153753443526168
64 0.16698350901957212
65 0.16804585634287617
66 0.1632747507000888
67 0.18593888697581878
68 0.19211931907774635
69 0.14415067487631977
70 0.16765862987500857
71 0.1946710743801653
72 0.1710465663395791
73 0.2005602607593592
74 0.16956531885344808

```
75 0.18652314049586777
76 0.18072168909227515
77 0.19075533699124983
78 0.20439450435568646
79 0.19356806229082718
80 0.12577556177856705
81 0.16516970987106228
82 0.11802929748271598
83 0.20883610712686598
84 0.1688767654000562
85 0.1849576573818998
86 0.16755232641971937
87 0.16449832661703437
88 0.1812035442811958
89 0.20558149109426346
90 0.18227256352022528
91 0.1660228126494092
92 0.15939395836653542
93 0.20383852157943075
94 0.22788925619834713
95 0.19131818181818183
96 0.1866735537190083
97 0.16797563107399754
98 0.16840019750472415
99 0.1904251504948475
100 0.17021027328127253
101 0.19888099173553725
102 0.18699182736455466
103 0.23665360422405884
104 0.16155706369248035
105 0.2466095270890726
106 0.17008552846268848
107 0.1863375573921029
108 0.21841942148760335
109 0.20872335298894276
110 0.19755452724085334
111 0.19086822393734487
112 0.15856556537598207
113 0.1921686466942149
114 0.20547693311780466
```

5.0.2 Effective Size

Rather similarly, you can compute the effective size of each node in a dictionary format, as shown below.

In the context of our dataset, this is a measure of how non-redundant a team's opponents are. For example, if a team plays against many teams who do not play each other, the node has a high

effective size. If a team plays against teams that mostly also play each other, it has a 'redundancy' in its competition and a low effective size.

[ ]:
```
effective_sizes = nx.effective_size(G_football)

for node_id, es in effective_sizes.items():
    print(f'{node_id} : {es}')
```

6 Conducting K-Core decomposition

In this module's assignment, you may need to conduct K-core decomposition.

First, however, we are going to introduce the next dataset you will be working with. This one is an undirected graph representing a community of dolphins, where each node represents a dolphin and each edge represents a friendship between dolphins. Each dolphin has the following attributes:

```
node [
  id 6
  label "DN21"
  smelliness 58
]
```

We will read in the dataset below.

[ ]:
```
G_dolphins = nx.read_gml("../../../assignments/source/NAModule1Assignment/
↪assets/dolphins.gml")
```

Now, to conduct K-core decomposition on the dolphins dataset, we will use the `nx.core_number` function. Similar to the constraint function, this also returns a dictionary. The key is the `id` of a dolphin, and the value is the highest value k of a k-core containing that dolphin.

[6]:
```
core_numbers = nx.core_number(G_dolphins)

for node_id, highest_core in core_numbers.items():
    print(f'{node_id} : {highest_core}')
```

```
Beak : 4
Beescratch : 4
Bumper : 3
CCL : 3
Cross : 1
DN16 : 3
DN21 : 4
DN63 : 4
Double : 4
Feather : 4
Fish : 4
Five : 1
Fork : 1
```

```
Gallatin : 4
Grin : 4
Haecksel : 4
Hook : 4
Jet : 4
Jonah : 4
Knit : 4
Kringel : 4
MN105 : 4
MN23 : 1
MN60 : 3
MN83 : 4
Mus : 3
Notch : 3
Number1 : 3
Oscar : 4
Patchback : 4
PL : 4
Quasi : 1
Ripplefluke : 2
Scabs : 4
Shmuddel : 3
SMN5 : 1
SN100 : 4
SN4 : 4
SN63 : 4
SN89 : 2
SN9 : 4
SN90 : 4
SN96 : 4
Stripes : 4
Thumper : 3
Topless : 4
TR120 : 2
TR77 : 4
TR82 : 1
TR88 : 2
TR99 : 4
Trigger : 4
TSN103 : 4
TSN83 : 2
Upbang : 4
Vau : 2
Wave : 2
Web : 4
Whitetip : 1
Zap : 4
Zig : 1
```

```
Zipfel : 2
```